

Week11_예습과제

[통계](#) [수정](#) [삭제](#)

카사바 · 방금 전

 0

Euron_2026-1



▼ 목록 보기

7/7



06 임베딩

Word2Vec

Word2Vec: 2013년 구글에서 공개한 임베딩 모델로 단어 간의 유사성을 측정하기 위해 **분포 가설**을 기반으로 개발됐다.

분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정. 단어 간의 동시 발생(co-occurrence) 확률 분포를 이용해 단어 간의 유사성을 측정한다.

e.g. '내일 자동차를 타고 부산에 간다'와 '내일 비행기를 타고 부산에 간다'라는 두 문장에서 '자동차'와 '비행기'는 주변에 분포한 단어들이 유사하므로 비슷한 의미일 것이라 예상함

이러한 가정을 통해 단어의 분산 표현을 학습할 수 있다.

분산 표현: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것

유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 된다. 이러한 방법으로 빈도 기반 벡터화에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계를 극복했으며, 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보를 효과적으로 표현한다.

단어 벡터화

단어를 벡터화하는 방법은 크게 희소 표현과 밀집 표현으로 나눌 수 있다. 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법은 희소 표현이며, Word2Vec은 밀집 표현이다.

표 6.3 단어의 희소 표현

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

빈도 기반 방법은 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비가 발생한다. 또한, 단어 간의 유사성을 반영하지 못하고, 벡터 간의 유사성을 계산하는 데도 많은 비용이 발생한다.

이에 비해 Word2Vec과 같은 밀집 표현은 단어를 고정된 크기의 실수 벡터로 표현하기 때문에 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않는다. 벡터 공간상에서 단어 간의 거리를 효과적으로 계산할 수 있으며, 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간을 활용할 수 있다.

밀집 표현 벡터화는 학습을 통해 단어를 벡터화하기 때문에 단어의 의미를 비교할 수 있다. 밀집 표현된 벡터를 단어 임베딩 벡터라고 한다.

Word2Vec은 밀집 표현을 위해 CBoW와 Skip-gram이라는 두 가지 방법을 사용한다.

CBoW

CBoW(Continuous Bag of Words): 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법

중심 단어는 예측해야 할 단어를 의미하며, 예측에 사용되는 단어들을 주변 단어라고 한다.

중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 정해야 하는데, 이 범위를 윈도우(Window)라고 한다. 이 윈도우를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습한다.

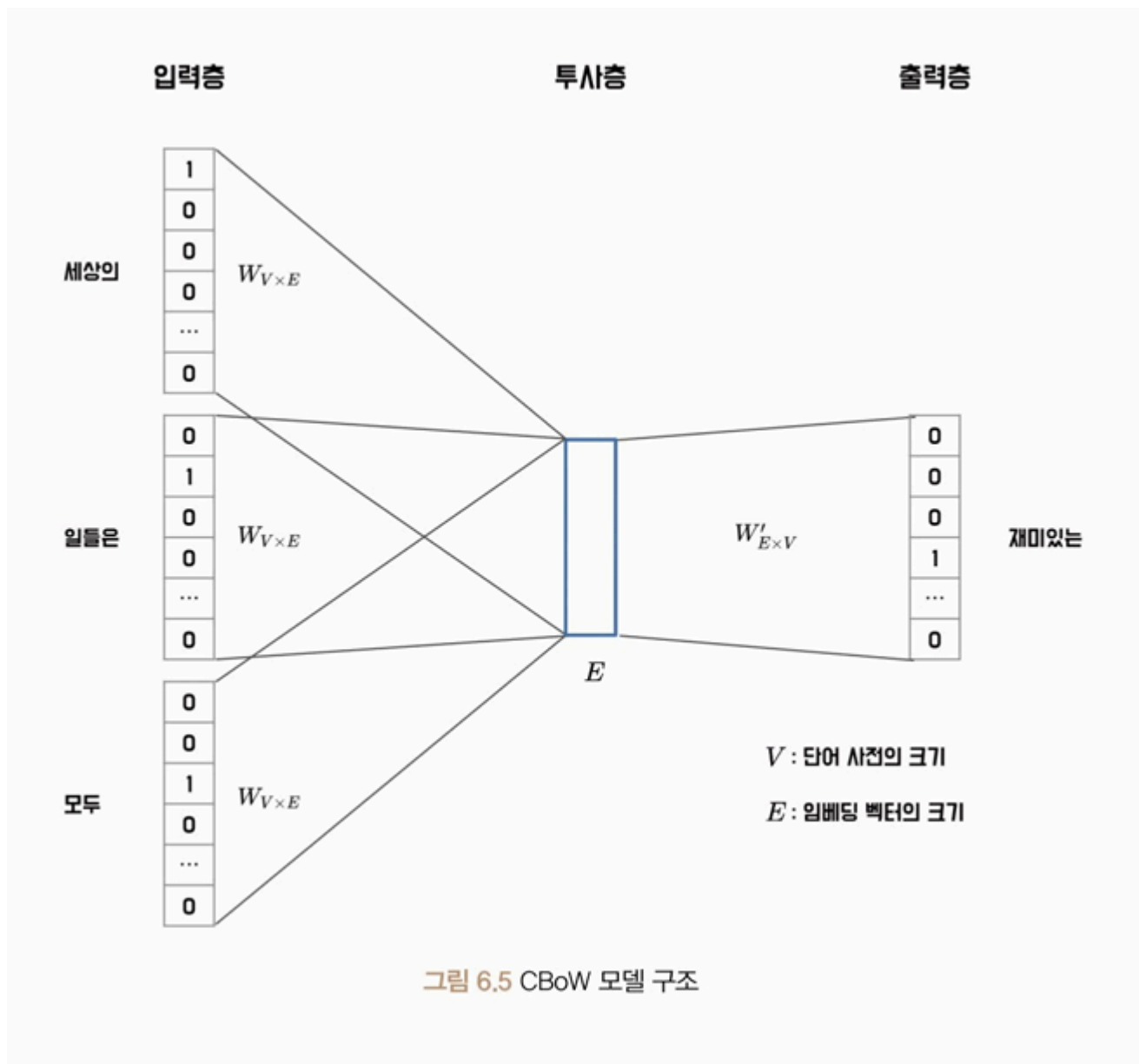
학습을 위해 윈도우를 이동해 가며 학습하는데, 이러한 방법을 슬라이딩 윈도우라고 한다. CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습할 수 있다.

다음 그림은 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터가 어떻게 구성되는지를 보여준다.

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

학습 데이터는 (주변 단어 | 중심 단어)로 구성된다. 이를 통해 대량의 말뭉치에서 효율적으로 단어의 분산 표현을 학습할 수 있다.



CBoW 모델은 각 입력 단어의 원-핫 벡터를 입력값으로 받는다. 입력 문장 내 모든 단어의 임베딩 벡터를 평균내어 중심 단어의 임베딩 벡터를 예측한다.

입력 단어는 원-핫 벡터로 표현돼 투사층에 입력된다. 투사층이란 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표 구조가 된다.

투사층을 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환된다.

계산된 평균 벡터를 가중치 행렬과 곱하면 V 크기의 벡터를 얻는다. 이 벡터에 소프트맥스 함수를 이용해 중심 단어를 예측한다.

Skip-gram

Skip-gram은 CBoW와 반대로 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델. 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만든다. 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습시킨다.

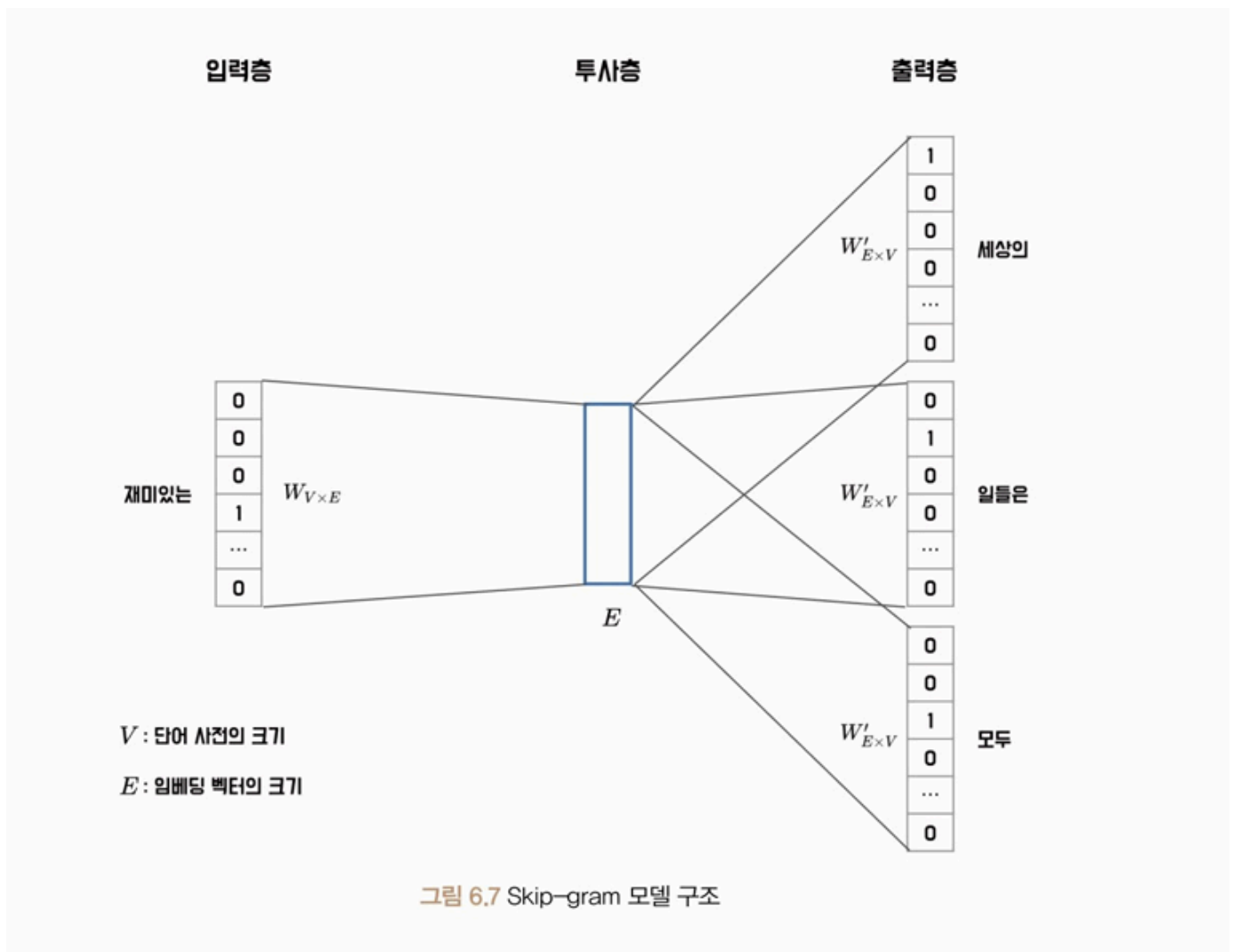
입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

Skip-gram과 CBoW는 학습 데이터 구성 방식에 차이가 있다. CBoW는 하나의 윈도우에서 하나의 학습 데이터가 만들어지는 반면 Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어진다.

이런 차이 때문에 Skip-gram은 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하므로 더 많은 학습 데이터셋을 추출할 수 있으며, 일반적으로 CBoW보다 더 뛰어난 성능을 보인다.

또한 Skip-gram은 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 되고 단어 벡터 공간에서 더 유의미한 거리 관계를 형성할 수 있다.



Skip-gram 모델도 CBoW와 마찬가지로 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져온다.

소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행한다. 말뭉치 크기가 커지면 필연적으로 단어 사전의 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 학습 속도가 느려지는 단점이 있다. 이를 해결하기 위해 계층적 소프트맥스와 네거티브 샘플링 기법을 적용해 학습 속도가 느려지는 문제를 완화한다.

계층적 소프트맥스

계층적 소프트맥스: 출력층을 이진 트리 구조로 표현해 연산을 수행한다. 자주 등장하는 단어일수록 트리의 상위 노드에 위치하고, 드물게 등장하는 단어일수록 하위 노드에 배치된다. 이런 식으로 확률을 계산하면 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성을 보인다.

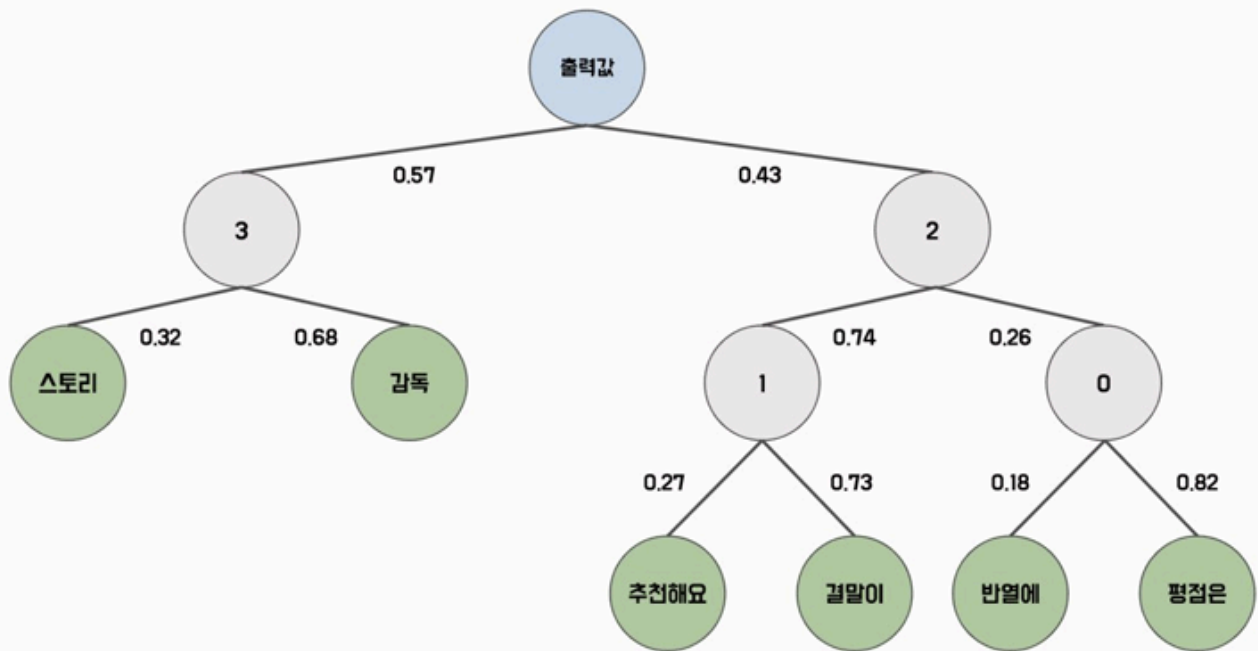


그림 6.8 계층적 소프트맥스 구조

각 노드는 학습이 가능한 벡터를 가지며, 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률을 계산한다.

앞 노드는 가장 깊은 노드로, 각 단어를 의미하며, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 한다. 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있다.

네거티브 샘플링

네거티브 샘플링: Word2Vec 모델에서 사용되는 확률적 샘플링 기법으로 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용한다.

학습 원도 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행한다. 이를 통해 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습할 수 있다.

이때 추출할 단어 n 은 일반적으로 5~20개를 사용하며, 각 단어가 추출될 확률을 다음과 같다.

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

네거티브 샘플링에서는 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 아니면 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류를 한다. 이를 위해 로지스틱 회귀 모델을 사용하며, 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습한다.

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

→

입력 데이터	출력 데이터
재미있는. 세상의	1
재미있는. 일들은	1
재미있는. 모두	1
재미있는. 걸어가다	0
재미있는. 주십시오	0
재미있는. 미리	0
재미있는. 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터



그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

네거티브 샘플링 모델에서는 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블 (0 or 1)을 가져와 내적 연산을 수행한다. 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환된다.

이때 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화된다.

모델 실습: Skip-gram

임베딩 클래스는 단어나 범주형 변수와 같은 이산 변수를 연속적인 벡터 형태로 변환해 사용할 수 있다. 연속적인 벡터 표현은 모델이 학습되는 동안 단어의 의미와 관련된 정보를 포착하고, 이를 기반으로 단어 간의 유사도를 계산한다.

이산 변수의 값을 연속적인 벡터로 변환하는 과정을 룩업이라 한다.

임베딩 클래스

```
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None,
    norm_type=2.0
)
```

```

from torch import nn

class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim
        )
        self.linear = nn.Linear(
            in_features=embedding_dim,
            out_features=vocab_size
        )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output

```

예제 6.4 영화 리뷰 데이터셋 전처리

```

import pandas as pd
from Korpora import Korpora
from konlpy.tag import Okt

corpus = Korpora.load("nsmc")
corpus = pd.DataFrame(corpus.test)

tokenizer = Okt()
tokens = [tokenizer.morphs(review) for review in corpus.text]
print(tokens[:3])

```

출력 결과

```

[['ㄱ', 'ㅋ'], ['GDNTOPCLASSINTHECLUB'], ['뭐', '야', '이', '평점', '들', '은', '....', '나쁜진',

```

예제 6.5 단어 사전 구축

```

from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

```

```

vocab = build_vocab(corpus=tokens, n_vocab=5000, special_tokens=["<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))

```

출력 결과

```

['<unk>', '.', '이', '영화', '의', '..', '가', '에', '...', '을']
5001

```

예제 6.6 Skip-gram의 단어 쌍 추출

```

def get_word_pairs(tokens, window_size):
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            center_word = sentence[idx]
            context_words = sentence[window_start:idx] + sentence[idx+1:window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs

word_pairs = get_word_pairs(tokens, window_size=2)
print(word_pairs[:5])

```

출력 결과

```

[['곤', 'ㅋ'], ['ㅋ', '곤'], ['뽕', '야'], ['뽕', '이'], ['야', '뽕']]

```

예제 6.7 인덱스 쌍 변환

```

def get_index_pairs(word_pairs, token_to_id):
    pairs = []
    unk_index = token_to_id["<unk>"]
    for word_pair in word_pairs:
        center_word, context_word = word_pair
        center_index = token_to_id.get(center_word, unk_index)
        context_index = token_to_id.get(context_word, unk_index)
        pairs.append([center_index, context_index])
    return pairs

```

```
index_pairs = get_index_pairs(word_pairs, token_to_id)
print(index_pairs[:5])
```

출력 결과

```
[[595, 100], [100, 595], [77, 176], [77, 2], [176, 77]]
```

예제 6.8 데이터로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

index_pairs = torch.tensor(index_pairs)
center_indexes = index_pairs[:, 0]
context_indexes = index_pairs[:, 1]

dataset = TensorDataset(center_indexes, context_indexes)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

예제 6.9 Skip-gram 모델 준비 작업

```
from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
word2vec = VanillaSkipgram(vocab_size=len(token_to_id), embedding_dim=128).to(device)
criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(word2vec.parameters(), lr=0.1)
```

예제 6.10 모델 학습

```
for epoch in range(10):
    cost = 0.0
    for input_ids, target_ids in dataloader:
        input_ids = input_ids.to(device)
        target_ids = target_ids.to(device)

        logits = word2vec(input_ids)
        loss = criterion(logits, target_ids)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    cost += loss

cost = cost / len(dataloader)
print(f"Epoch : {epoch+1:4d}, Cost : {cost:.3f}")
```

출력 결과

```
Epoch :    1, Cost : 6.196
Epoch :    2, Cost : 5.980
Epoch :    3, Cost : 5.931
Epoch :    4, Cost : 5.901
Epoch :    5, Cost : 5.878
Epoch :    6, Cost : 5.861
Epoch :    7, Cost : 5.846
Epoch :    8, Cost : 5.833
Epoch :    9, Cost : 5.822
Epoch :   10, Cost : 5.812
```

예제 6.11 임베딩 값 추출

```
token_to_embedding = dict()
embedding_matrix = word2vec.embedding.weight.detach().cpu().numpy()

for word, embedding in zip(vocab, embedding_matrix):
    token_to_embedding[word] = embedding

index = 30
token = vocab[index]
token_embedding = token_to_embedding[token]
print(token)
print(token_embedding)
```

출력 결과

연기

```
[ -1.3700286  -0.53419936 -0.4979807  -0.2555711  -0.87646496 -0.43019596
   1.5725858   1.0621762   0.8001532  -0.10356946  2.5778508  -0.37419274
  -0.17541623 -0.10778847 -0.6680416  -1.6015458  -0.34703737  0.93376935
   0.7599935  -0.25358564  1.8083229   1.2734787  -0.45371383 -1.4026566
  -1.0142812  -0.56605715 -1.2538975  -0.28078136 -0.29474387 -0.73899305
  -0.8260153  -1.3297142  -0.42350522 -1.8581526  -1.0632359   0.77626246
   0.6720073  -0.33520705  0.15288073 -1.640648   -0.15055683 -1.0484744
  -0.5355086  -0.35930106 -0.7261805  -0.12321096 -0.44906664 -1.106636
   0.08471    1.2125494  -0.4449702  -0.04542563 -0.0528081   0.23515376
  -0.35838902 -0.84893566 -1.1192309   1.1554664   0.16187583  1.9194432
   0.61413866  0.37279838 -0.21057443  0.91979414 -0.4715557   0.10846171
  -0.26331607 -1.4180142   0.5514552  -1.3333958   0.82356685  0.81480145
  -1.1805079  -0.2580851   1.1900429  -1.3635898   0.03030477 -1.7568526
   0.23030454  1.7480062  -0.23844935 -1.2764553   0.03480529 -0.99042135
  -1.3860449  -0.6464405  -0.86604947 -0.50180215  0.51322734 -0.849864
   0.73899335  0.21484916 -0.3517042  -0.15372042 -1.0275272   0.9631892
  -0.00993367 -0.19878528 -0.81149644 -0.0892992  -1.7576708  -0.18071395
   0.95879394 -0.03787694  0.32819244  0.5787388  -1.3274959  -0.77856725
   1.1196953  -0.704148   -0.66877955  0.83742994 -0.94897085 -0.1719094
   0.16892032 -0.68807644  0.5901872  -1.3861064  -0.69723094 -1.3477777
   1.6631497  -0.27955115 -0.8802952  -0.8985007   0.7169123   0.81557995
   0.4633692   0.6892731 ]
```

Word2Vec 모델의 임베딩 행렬을 이용해 각 단어의 임베딩 값을 매핑하고, 인덱스 30 값의 단어와 임베딩 값을 출력한다. 이 임베딩 값으로 단어 간의 유사도를 확인할 수 있다. 임베딩의 유사도를 측정할 때는 코사인 유사도가 가장 일반적으로 사용되는 방법이다.

코사인 유사도는 두 벡터 간의 각도를 이용하여 유사도를 계산하며, 두 벡터가 유사할수록 값이 1에 가까워지고, 다를수록 0에 가까워진다.

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

a와 b는 유사도를 계산하려는 벡터이며, 두 벡터를 내적인 벡터의 크기의 곱을 나눠 코사인 유사도를 계산할 수 있다. 벡터의 크기는 각 성분의 제곱합에 루트를 씌운 값이다.

예제 6.12 단어 임베딩 유사도 계산

```
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a) / (norm(b, axis=1) * norm(a))
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1]
    return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도 : {cosine_matrix[index]:.4f}")
```

출력 결과

```
연기와 가장 유사한 5 개 단어
편집 - 유사도 : 0.3308
배우 - 유사도 : 0.3110
할듯 - 유사도 : 0.3109
```

주연 - 유사도 : 0.3057
높다 - 유사도 : 0.2917

모델 실습: Gensim

젠심 라이브러리를 활용하면 Word2Vec과 같은 자연어 처리 모델을 쉽게 구성할 수 있다. 젠심 라이브러리는 대용량 텍스트 데이터의 처리를 위한 효율적인 방법을 제공해 대규모 데이터셋에서도 효과적으로 모델을 학습할 수 있다.

Word2Vec 클래스

```
word2vec = gensim.models.Word2Vec(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000  
)
```

예제 6.13 Word2Vec 모델 학습

```
import pandas as pd  
from Korpora import Korpora  
from konlpy.tag import Okt  
  
corpus = Korpora.load("nsmc")  
corpus = pd.DataFrame(corpus.test)  
  
tokenizer = Okt()  
tokens = [tokenizer.morphs(review) for review in corpus.text]  
  
from gensim.models import Word2Vec
```

```
word2vec = Word2Vec(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=1,
    sg=1,
    epochs=3,
    max_final_vocab=10000
)

word2vec.save("../models/word2vec.model")
word2vec = Word2Vec.load("../models/word2vec.model")
```

예제 6.14 임베딩 추출 및 유사도 계산

```
word = "연기"
print(word2vec.wv[word])
print(word2vec.wv.most_similar(word, topn=5))
print(word2vec.wv.similarity(w1=word, w2="연기력"))
```

출력 결과

```
[ -0.06621774 -0.36674547  0.1663604   0.58909315  0.0372135  -0.02143276
   0.17337418 -0.16976535 -0.6349239   0.42311475 -0.10692125 -0.22684318
  -0.16229197 -0.05722928 -0.35821962 -0.07017373 -0.3062584   0.47364476
  -0.31980535  0.35628077  0.69763774  0.5071293  -0.32322797 -0.04212112
  -0.23542376 -0.02514695 -0.5616551   0.02662417  0.29563123  0.00791464
  -0.44979006  0.26099336  0.3812289  -0.12991887  0.09320281 -0.24161798
   0.18620354 -0.34542528 -0.2328435  -0.3889403   0.14078872  0.373868
  -0.10626898 -0.48218638 -0.28256202  0.1195678  -0.09368916 -0.3011636
   0.13934779  0.16056955  0.9585552   0.5883294  -0.00941139  0.41810927
  -0.4461008  -0.29347804  0.10102902  0.14089915 -0.14858654  0.13934498
  -0.06318886  0.08243572  0.18954007  0.19416073 -0.20150624  0.16989756
  -0.08909222  0.08483594  0.0566133  -0.12495491 -0.19386682 -0.30217433
  -0.29095522  0.14005096 -0.04750573  0.06570946 -0.21700248 -0.30987862
  -0.18673119  0.5145331   0.07081667 -0.08470267  0.21554314  0.8316306
   0.36727914  0.23544413  0.13670594 -0.33239782  0.27047485  0.06056395
  -0.03666366  0.09182108  0.1179811  -0.0013674   0.21825102 -0.10470655
  -0.1856406  -0.09957626 -0.09391525 -0.32195768 -0.1735274  -0.19358248
   0.07429539 -0.39614916 -0.03144876 -0.00670219  0.55012363  0.3297665
  -0.14430706 -0.382958   0.40008786 -0.0077886  -0.01534699  0.24336195
   0.27449661 -0.19288298  0.3035119   0.2860486  -0.00114569  0.40013826
  -0.04160194 -0.08082829  0.06571459 -0.0471548   0.01440274  0.11111417
  -0.4780807  -0.2376135 ]
(['연기력', 0.7710634469985962), ('캐스팅', 0.7409313321113586), ('몸매', 0.696537971496582), ('(
0.77106357
```


fastText: 2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델로, 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘. 단어와 문장을 벡터로 변환하는 기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용된다.

이런 벡터화 기술은 Word2Vec과 유사하지만, fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능을 제공한다. 하위 단어를 고려하기 위해 N-gram을 사용해 단어를 분해하고 벡터화하는 방법으로 동작한다.

기호가 추가된 N-gram을 사용하여 하위 단어 집합으로 분해된다.

분해된 하위 단어 집합에서 나뉘지지 않은 단어 자신도 포함되며, 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 된다.

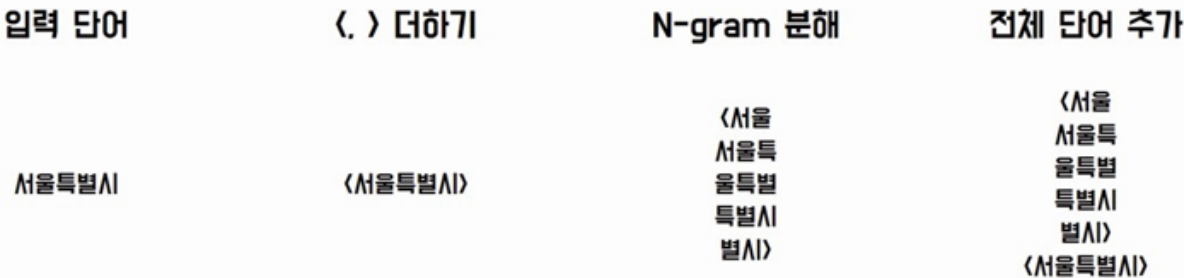


그림 6.11 fastText의 하위 단어 집합

fastText는 위와 같은 방법으로 하위 단어 집합을 생성한다. 입력 단어를 하위 단어로 분해하면 총 6개의 하위 단어가 존재하게 된다. 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산한다.

모델 실습

FastText 클래스

```
fasttext = gensim.models.FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

예제 6.15 KorNLI 데이터셋 전처리

```
from Korpora import Korpora  
  
corpus = Korpora.load("kornli")  
corpus_texts = corpus.get_all_texts() + corpus.get_all_pairs()  
tokens = [sentence.split() for sentence in corpus_texts]  
  
print(tokens[:3])
```

출력 결과

```
[['개념적으로', '크림', '스키밍은', '제품과', '자리라는', '두', '가지', '기본', '차원을', '가지고', '있고']  
[...]  
[...]]
```

예제 6.16 fastText 모델 실습

```
from gensim.models import FastText  
  
fastText = FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

```

    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=5,
    sg=1,
    max_final_vocab=20000,
    epochs=3,
    min_n=2,
    max_n=6
)

# fastText.save("../models/fastText.model")
# fastText = FastText.load("../models/fastText.model")

```

예제 6.17 fastText OOV 처리

```

oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))

```

출력 결과

```

False
[('사랑', 0.8724774122238159), ('사랑에', 0.8292666673660278), ('사랑의', 0.7907654643058777), ('

```

순환 신경망

순환 신경망: 순서가 있는 연속적인 데이터를 처리하는 데 적합한 구조를 갖고 있다. 순환 신경망은 각 시점의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동한다.

자연어 데이터는 연속적인 데이터의 일종으로 볼 수 있다.

예시는 단어들의 연속으로 이루어진 자연어 데이터가 가지는 특징을 잘 보여준다. 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추할 수 있다.

순환 신경망

순환 신경망: 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류. 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용된다.

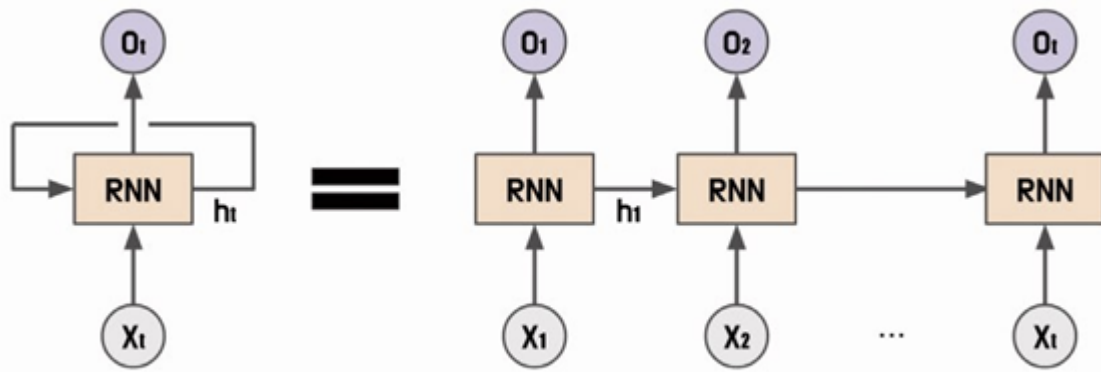


그림 6.12 순환 신경망의 셀

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

일대다 구조

일대다 구조: 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조

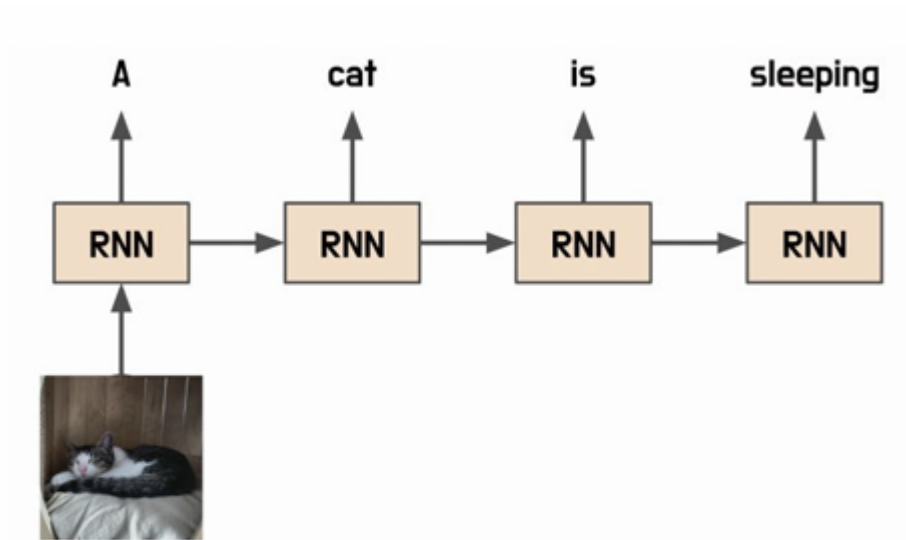


그림 6.13 순환 신경망의 일대다 구조

다대일 구조

다대일 구조: 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조

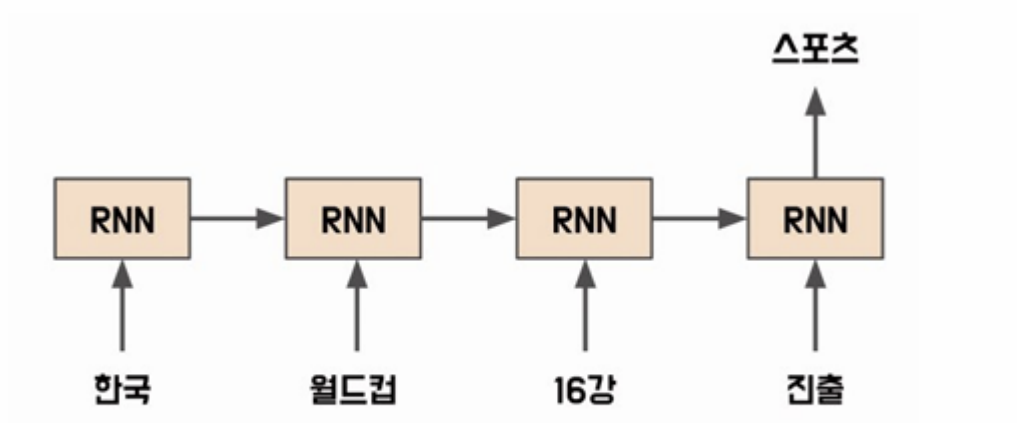


그림 6.14 순환 신경망의 다대일 구조

다대다 구조

다대다 구조: 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조

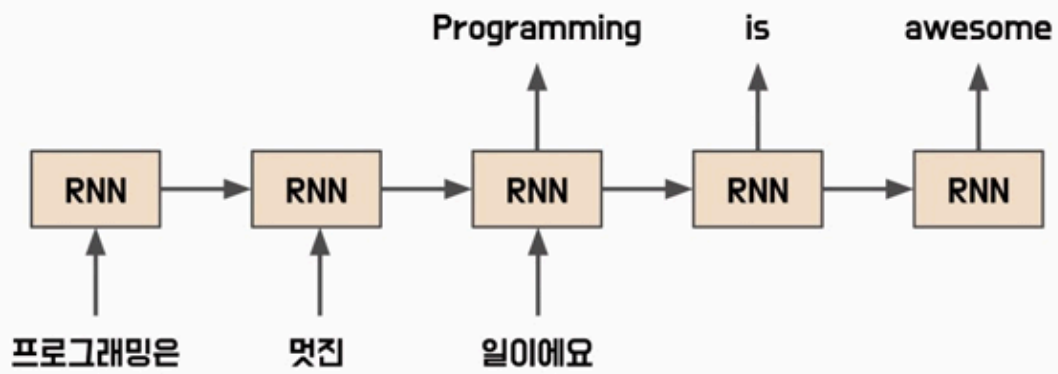


그림 6.15 순환 신경망의 다대다 구조

양방향 순환 신경망

양방향 순환 신경망: 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식

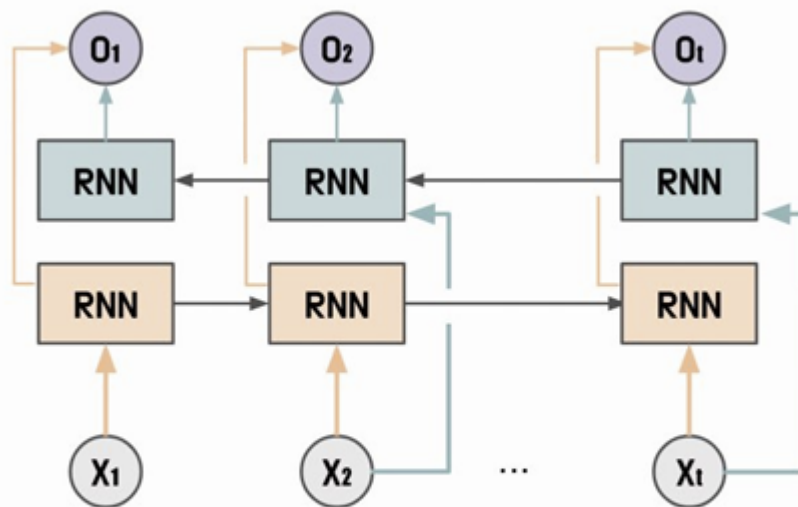


그림 6.16 양방향 순환 신경망 구조

다층 순환 신경망

다층 순환 신경망: 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계됨

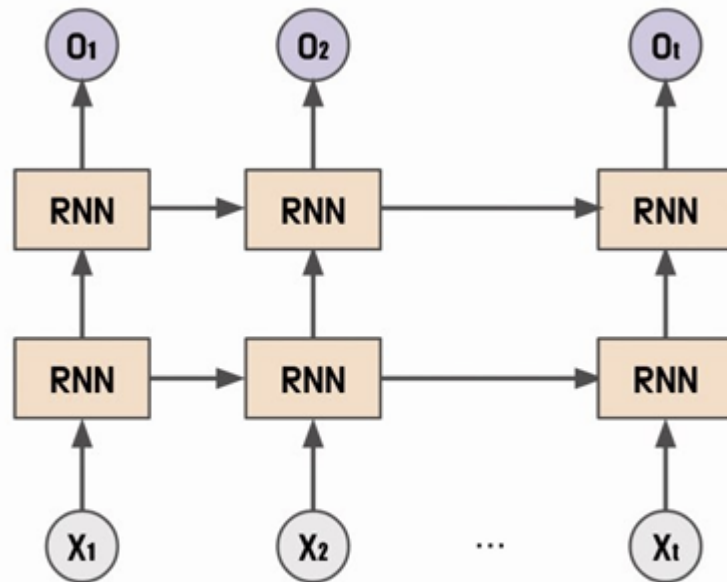


그림 6.17 다중 순환 신경망 구조

순환 신경망 클래스

순환 신경망 클래스

```
rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1,
    nonlinearity="tanh",
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False
)
```

예제 6.18 양방향 다층 신경망

```
import torch
from torch import nn
```

```

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape)
print(hidden.shape)

```

출력 결과

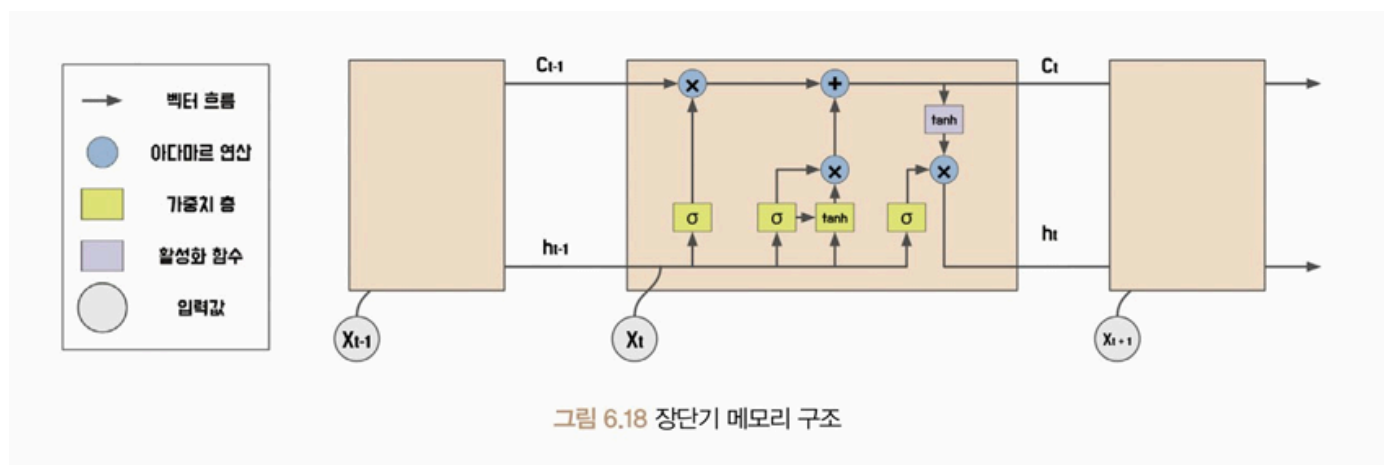
```

torch.Size([4, 6, 512])
torch.Size([6, 4, 256])

```

장단기 메모리

장단기 메모리(LSTM): 1997년 셉 호흐라이터와 유르겐 슈미트후버가 제안한 알고리즘으로 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 문제. 이를 해결하기 위해 장단기 메모리를 사용한다. 장단기 메모리는 순환 신경망과 비슷한 구조를 가지지만, 메모리 셀과 게이트라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제를 해결한다.



장단기 메모리 구조

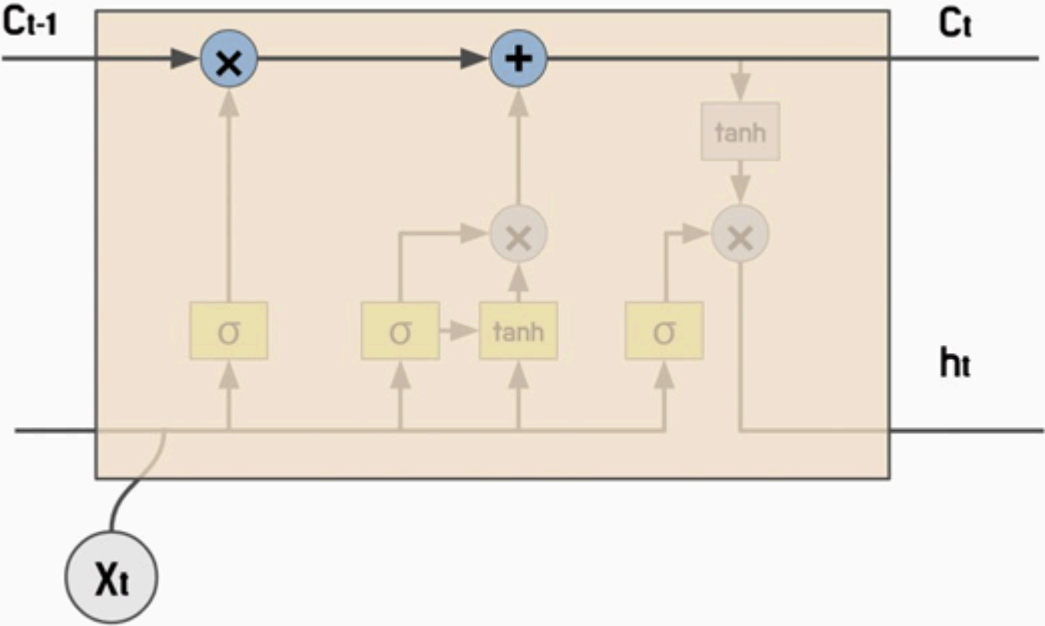


그림 6.19 장단기 메모리 셀 상태

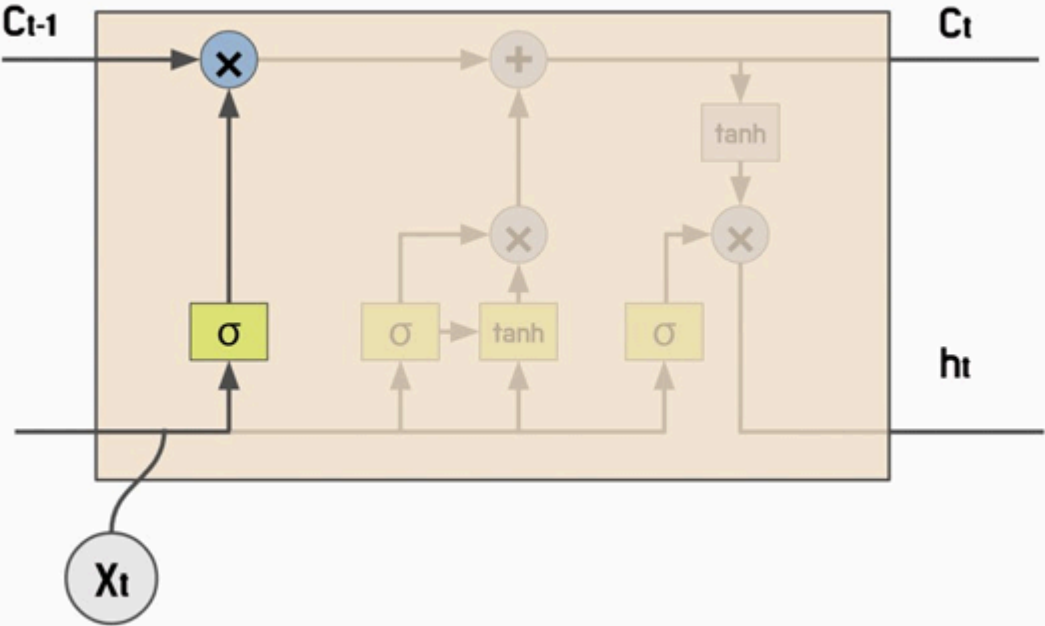


그림 6.20 장단기 메모리의 망각 게이트

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

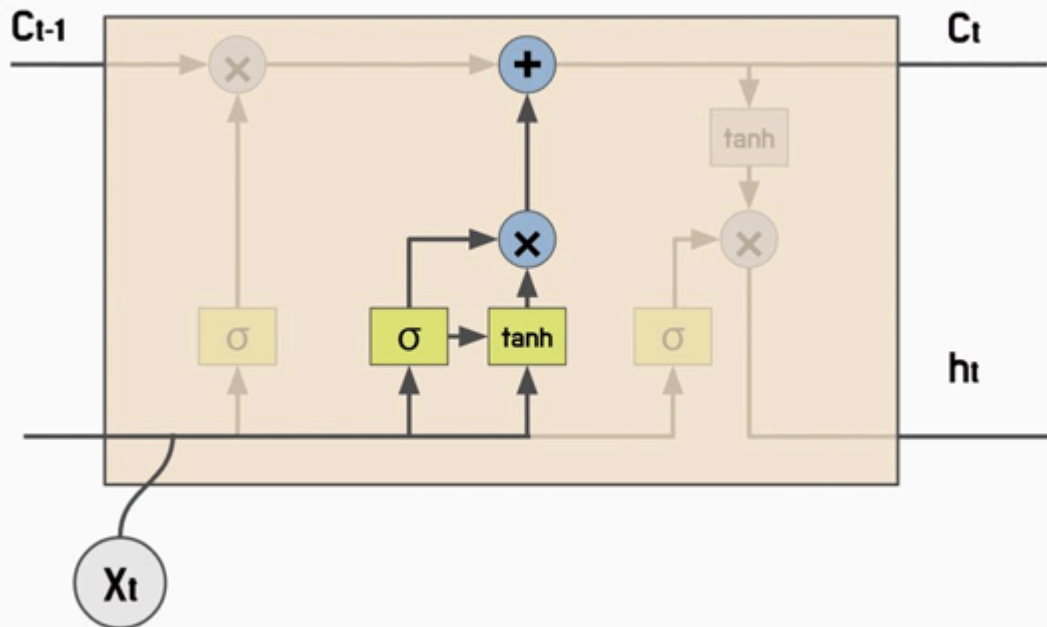


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_t \odot i_t$$

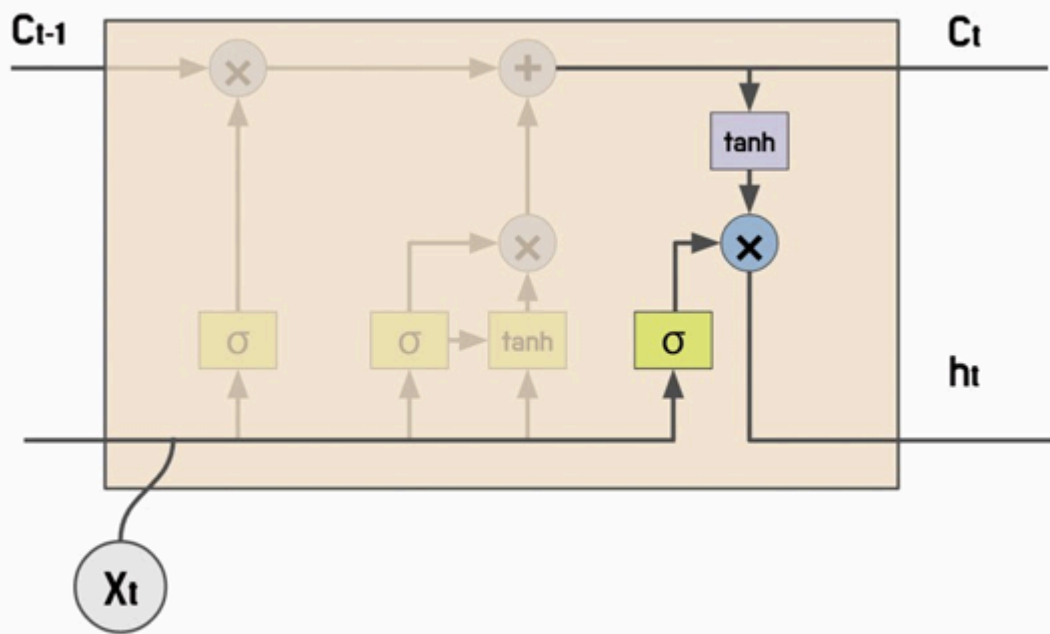


그림 6.22 장단기 메모리의 출력 게이트

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

장단기 메모리 클래스

장단기 메모리 클래스

```
lstm = torch.nn.LSTM(  
    input_size,  
    hidden_size,  
    num_layers=1,  
    bias=False,  
    batch_first=True,  
    dropout=0,  
    bidirectional=False,  
    proj_size=0  
)
```

예제 6.19 양방향 다층 장단기 메모리

```
import torch  
from torch import nn  
  
input_size = 128  
output_size = 256  
num_layers = 3  
bidirectional = True  
proj_size = 64  
  
model = nn.LSTM(  
    input_size=input_size,  
    hidden_size=output_size,  
    num_layers=num_layers,  
    batch_first=True,  
    bidirectional=bidirectional,  
    proj_size=proj_size,  
)  
  
batch_size = 4
```

```

sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else output_size,
)
c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

print(outputs.shape)
print(h_n.shape)
print(c_n.shape)

```

출력 결과

```

torch.Size([4, 6, 128])
torch.Size([6, 4, 64])
torch.Size([6, 4, 256])
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/rnn.py:1141: UserWarning: LSTM with p
result = _VF.lstm(

```

모델 실습

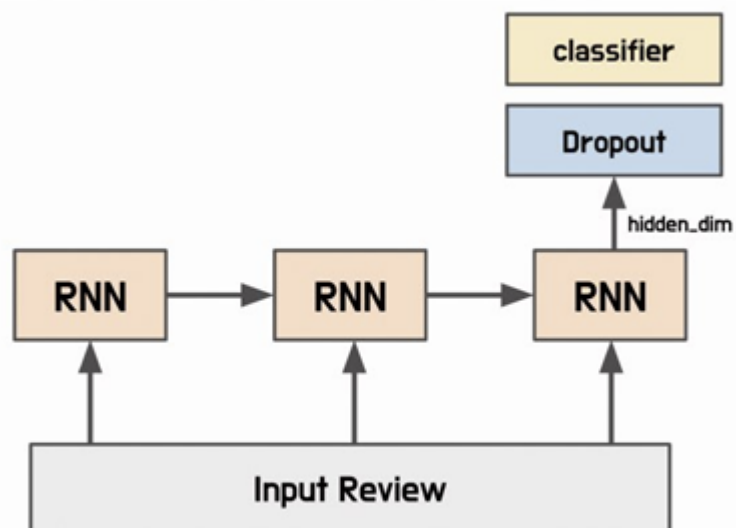


그림 6.23 긍/부정 분류 모델 구조

```

from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm"
    ):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )
        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        if bidirectional:
            self.classifier = nn.Linear(hidden_dim * 2, 1)
        else:
            self.classifier = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        output, _ = self.model(embeddings)
        last_output = output[:, -1, :]
        last_output = self.dropout(last_output)
        logits = self.classifier(last_output)
        return logits

```

```

import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))

```

출력 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을 손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
 해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 **description** 을 참고,
 해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```

# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: http://ai.stanford.edu/~amaas/data/sentiment/

# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/

```

```

[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_train.txt
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_test.txt

```

```

|      | text
|-----|:-----
| 33553 | 모든 편견을 날려 버리는 가슴 따뜻한 영화. 로버트 드 니로, 필립 세이모어 호프만 영원하라. |
|  9427 | 무한 리메이크의 소재. 감독의 역량은 항상 그 자리에... |
|   199 | 신날 것 없는 애니.
| 12447 | 잔잔 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
Training Data Size : 45000
Testing Data Size : 5000

```

예제 6.22 데이터 토큰화 및 단어 사전 구축

```
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

출력 결과

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
5002
```

예제 6.23 정수 인코딩 및 패딩

```
import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]
```



```
]
```

```
max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])
```

출력 결과

```
[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]
```

예제 6.24 데이터로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

예제 6.25 손실 함수와 최적화 함수 정의

```
from torch import optim

n_vocab = len(token_to_id)
hidden_dim = 64
embedding_dim = 128
n_layers = 2

device = "cuda" if torch.cuda.is_available() else "cpu"
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim, n_layers=n_layers
).to(device)
```

```
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)
```

예제 6.26 모델 학습 및 테스트

```
def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if step % interval == 0:
            print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits)>.5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)
```

출력 결과

```

Train Loss 0 : 0.6866534948348999
Train Loss 500 : 0.6823862150995555
Train Loss 1000 : 0.6510271433885996
Train Loss 1500 : 0.6200004250585517
Train Loss 2000 : 0.5973543037419793
Train Loss 2500 : 0.5795147728021505
Val Loss : 0.46141017963901493, Val Accuracy : 0.7792
Train Loss 0 : 0.33064547181129456
Train Loss 500 : 0.4490690616492978
Train Loss 1000 : 0.4407945908643268
Train Loss 1500 : 0.44122220700399467
Train Loss 2000 : 0.43353727353789817
Train Loss 2500 : 0.4303044144038866
Val Loss : 0.42143663623557687, Val Accuracy : 0.8098
Train Loss 0 : 0.3195874094963074
Train Loss 500 : 0.3698648054323748
Train Loss 1000 : 0.37224400674665603
Train Loss 1500 : 0.3698391514438776
Train Loss 2000 : 0.37033938994442206
Train Loss 2500 : 0.37080571157593384
Val Loss : 0.399040534235418, Val Accuracy : 0.8164
Train Loss 0 : 0.2371215522289276
Train Loss 500 : 0.3325901468565126
Train Loss 1000 : 0.3309891494539949
Train Loss 1500 : 0.33518812008733195
Train Loss 2000 : 0.33802291982505156
Train Loss 2500 : 0.3367714855860158
Val Loss : 0.3956116630722539, Val Accuracy : 0.8206
Train Loss 0 : 0.2683178782463074
Train Loss 500 : 0.29978999038210175
Train Loss 1000 : 0.29925538034929144
Train Loss 1500 : 0.30125387554999433
Train Loss 2000 : 0.30419627348634076
Train Loss 2500 : 0.30463439633981176
Val Loss : 0.4044163741718847, Val Accuracy : 0.821

```

예제 6.27 학습된 모델로부터 임베딩 추출

```

token_to_embedding = dict()
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()

for word, emb in zip(vocab, embedding_matrix):
    token_to_embedding[word] = emb

token = vocab[1000]
print(token, token_to_embedding[token])

```

출력 결과

```

보고싶다 [ 0.3060669  0.405856 -0.6062377  0.26378956 -0.49522546  0.1393255
-1.0710139  1.7116443  0.19716448 -0.27249926  1.4898953 -0.9961566
-0.57864934  0.5247539  0.3181274  0.85601634 -0.3267693 -0.15991402
 1.4392368  0.7507427  0.6704012 -1.6193398  0.05843502  2.1680667
 0.27619565  0.35380408  0.7119952 -0.2312717 -1.2261006 -0.48431394

```

```

-0.6124159  0.8361179 -0.16714838 -1.818825  -0.01270011  1.4118593
 1.4019202 -3.304726  0.645091  1.5672307 -0.024147 -0.29292262
 2.2480469 -1.9909304 -0.1897206 -0.05372866  0.7788252 -0.74750024
 0.5640589  0.37150913  0.51749355  0.6654714 -0.06851145  0.07796954
 1.6684883  1.5484782 -1.0300803 -0.00962836  0.3480971 -0.30097073
 0.62488735 -2.515338  0.5363361  0.18829024  2.2370698 -0.6247874
-1.2965727  0.97360224 -0.9887784 -0.21572527 -0.82928735 -3.0779815
-0.10086456 0.27960685 -0.45951498 0.61698633 0.05810185 -0.3953985
-0.69636995 1.0007434 -0.2281954 -1.1285896 0.3969117 -0.5400716
 1.4059969 0.7892443 0.611267 0.28321654 1.3453951 0.88394874
 1.3218311 0.83737385 0.6385697 -2.2994955 0.716826 -0.8297346
-0.9310922 0.8405451 -0.03638813 -0.56758803 -0.8356645 -1.4194324
-0.37329882 1.0024283 -1.1949705 -0.743632 0.5874541 0.32547578
 0.2913848 -0.14602208 1.1510195 -1.9079098 -0.4842883 0.24554822
 0.21113078 -0.04248791 -0.23589022 -0.9171441 -0.5523578 1.8296788
-0.2447228 0.4293891 -1.2398363 -2.067702 -0.01413966 0.7759506
 0.09090654 -0.45518965]

```

예제 6.29 사전 학습된 임베딩 계층 적용

```

import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm",
        pretrained_embedding=None
    ):
        super().__init__()
        if pretrained_embedding is not None:
            self.embedding = nn.Embedding.from_pretrained(
                torch.tensor(pretrained_embedding, dtype=torch.float32)
            )
        else:
            self.embedding = nn.Embedding(
                num_embeddings=n_vocab,
                embedding_dim=embedding_dim,
                padding_idx=0
            )

        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,

```

```

    )
elif model_type == "lstm":
    self.model = nn.LSTM(
        input_size=embedding_dim,
        hidden_size=hidden_dim,
        num_layers=n_layers,
        bidirectional=bidirectional,
        dropout=dropout,
        batch_first=True,
    )

    if bidirectional:
        self.classifier = nn.Linear(hidden_dim * 2, 1)
    else:
        self.classifier = nn.Linear(hidden_dim, 1)
    self.dropout = nn.Dropout(dropout)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    output, _ = self.model(embeddings)
    last_output = output[:, -1, :]
    last_output = self.dropout(last_output)
    logits = self.classifier(last_output)
    return logits

```

예제 6.30 사전 학습된 임베딩을 사용한 모델 학습

```

from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim,
    n_layers=n_layers, pretrained_embedding=init_embeddings
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

출력 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 **description** 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39
```

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: <http://ai.stanford.edu/~amaas/data/sentiment/>

```
# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_train.txt
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_test.txt
```

```
|      | text
|-----:|:-----
| 33553 | 모든 편견을 날려 버리는 가슴 따뜻한 영화. 로버트 드 니로, 필립 세이모어 호프만 영원하라. |
|  9427 | 무한 리메이크의 소재. 감독의 역량은 항상 그 자리에...
|   199 | 신날 것 없는 애니.
| 12447 | 잔잔 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
```

Training Data Size : 45000

Testing Data Size : 5000

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
```

5002

```
[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]
```

Train Loss 0 : 0.7048603892326355

```
/tmp/ipykernel_1395/449981664.py:87: UserWarning: To copy construct from a tensor, it is recom
train_ids = torch.tensor(train_ids)
```

```
/tmp/ipykernel_1395/449981664.py:88: UserWarning: To copy construct from a tensor, it is recom
test_ids = torch.tensor(test_ids)
```

Train Loss 500 : 0.6927574432776598

Train Loss 1000 : 0.657947656038877

Train Loss 1500 : 0.6163621224577153

Train Loss 2000 : 0.5887107210329686

Train Loss 2500 : 0.5680219356666227

Val Loss : 0.47720152615739136, Val Accuracy : 0.7822

Train Loss 0 : 0.3823622167110443

Train Loss 500 : 0.4712985908378384

Train Loss 1000 : 0.4719334789714613

Train Loss 1500 : 0.473223712019806

Train Loss 2000 : 0.4686457407751541

Train Loss 2500 : 0.46839252571578693

Val Loss : 0.4333950311135941, Val Accuracy : 0.7992

Train Loss 0 : 0.2568286061286926

Train Loss 500 : 0.46342676402090077

```
Train Loss 1000 : 0.4606587705197749
Train Loss 1500 : 0.4526384821401128
Train Loss 2000 : 0.4522544730832611
Train Loss 2500 : 0.44964724538422546
Val Loss : 0.48321028963064616, Val Accuracy : 0.7472
Train Loss 0 : 0.5696015357971191
Train Loss 500 : 0.44430506746925996
Train Loss 1000 : 0.43935377033916745
Train Loss 1500 : 0.44222024474280586
Train Loss 2000 : 0.436171752979224
Train Loss 2500 : 0.4362681849134631
Val Loss : 0.45890403836489485, Val Accuracy : 0.7604
Train Loss 0 : 0.504827618598938
Train Loss 500 : 0.4312875662110523
Train Loss 1000 : 0.42549178704664065
Train Loss 1500 : 0.42183705304143115
Train Loss 2000 : 0.4230184424435836
Train Loss 2500 : 0.42796811345266084
Val Loss : 0.41058064602053584, Val Accuracy : 0.812
```

합성곱 신경망

합성곱 신경망: 주로 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 해석하기 위해 사용되는
인공 신경망의 한 종류

합성곱 계층 클래스

2차원 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0,
    dilation=1,
    groups=1,
    bias=True,
    padding_mode="zeros"
)
```

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$

$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

활성화 맵

활성화 맵: 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값

풀링



그림 6.27 최댓값 풀링

풀링 클래스

2차원 최대값 풀링 클래스

```
pool = torch.nn.MaxPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    dilation=1  
)
```

2차원 평균값 풀링 클래스

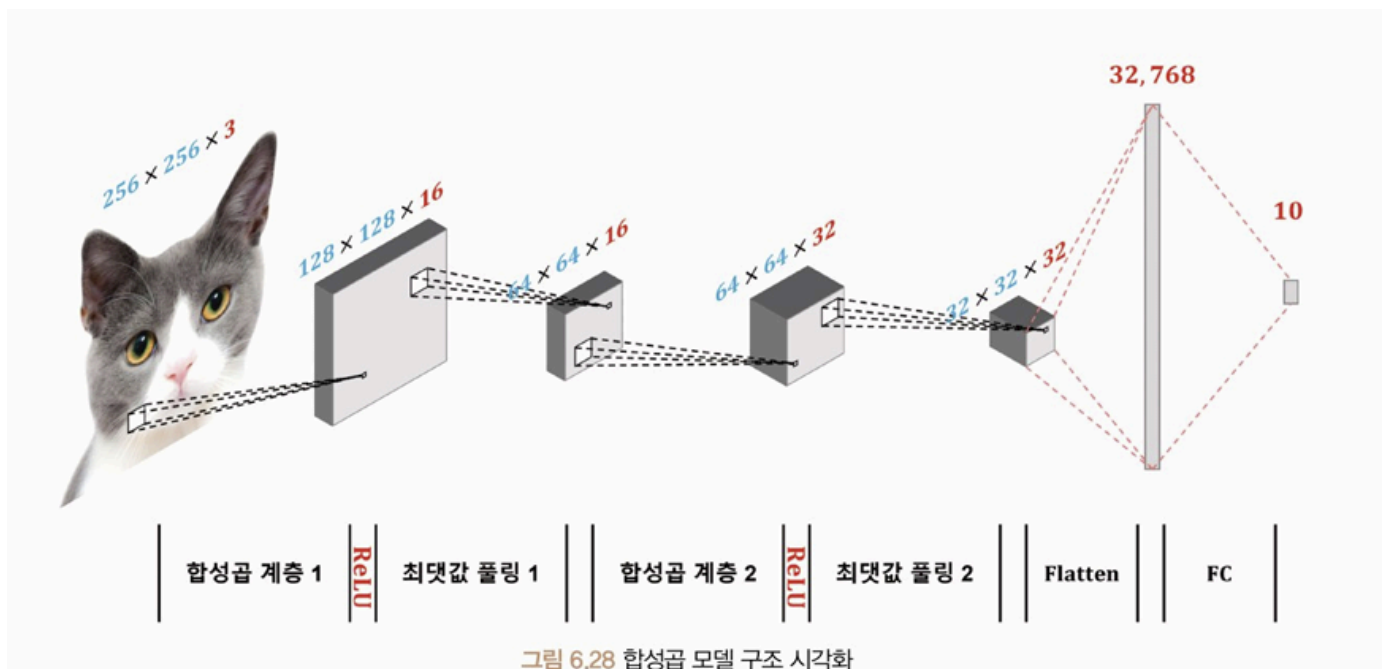
```
pool = torch.nn.AvgPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    count_include_pad=True  
)
```

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층

완전 연결 계층: 각 입력 노드가 모든 출력 노드와 연결된 상태



예제 6.31 합성곱 모델

```
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.fc = nn.Linear(32 * 32 * 32, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = torch.flatten(x)
```

```
x = self.fc(x)
return x
```

모델 실습

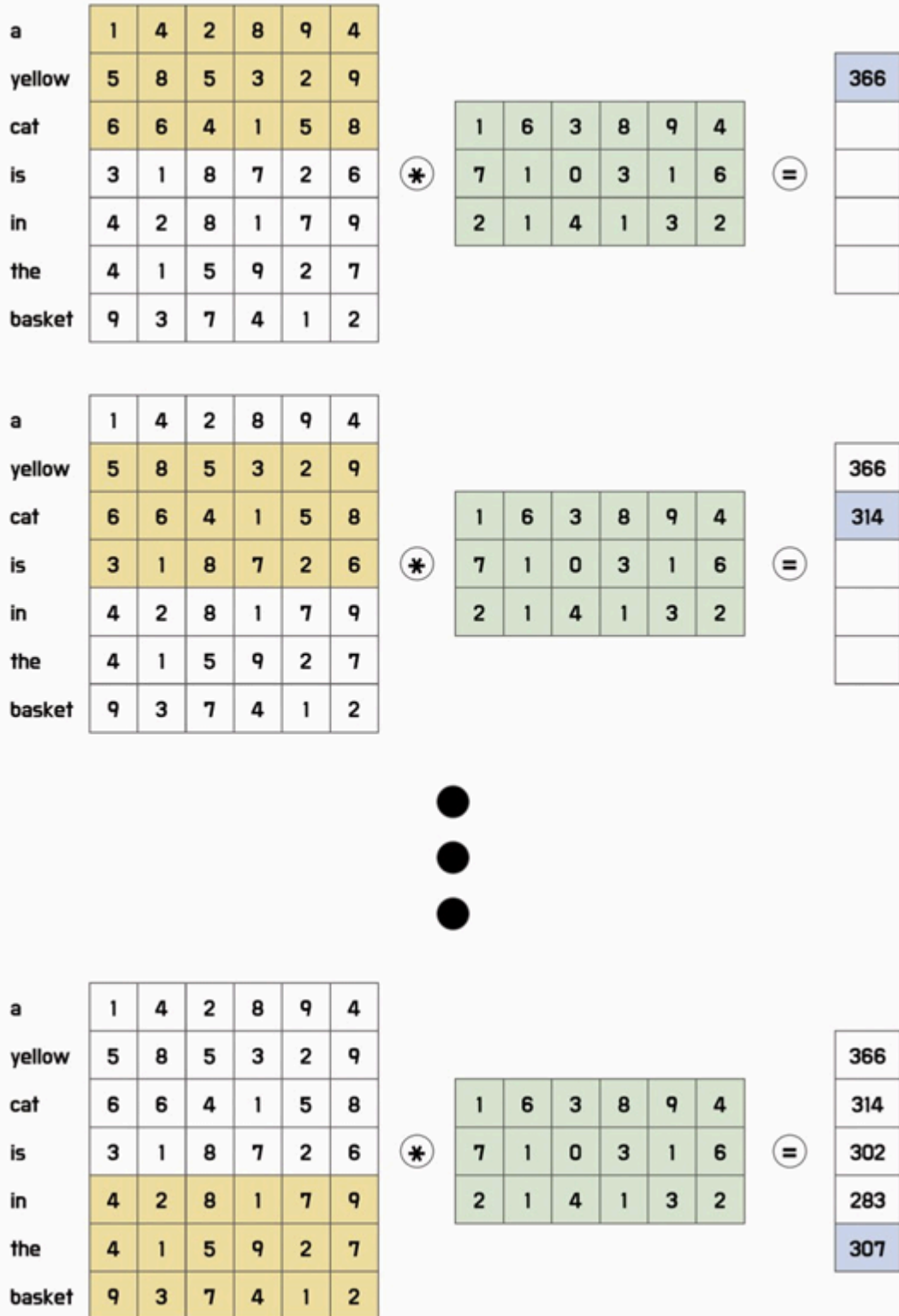


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

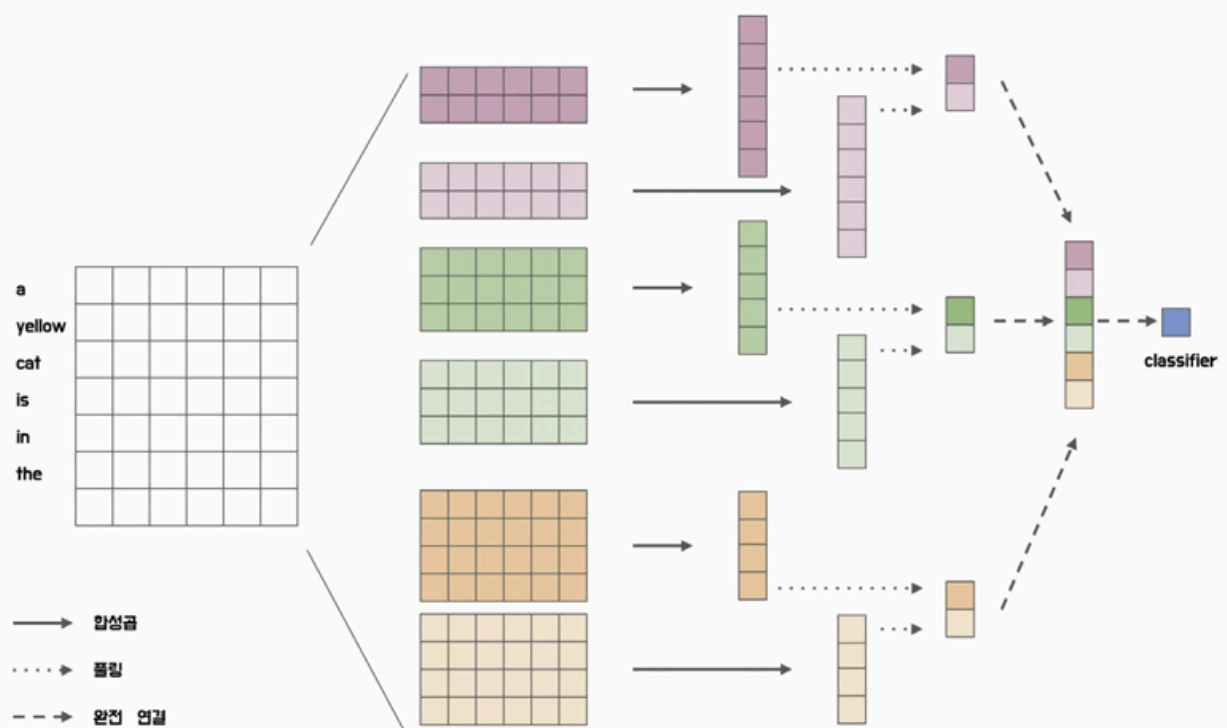


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산

예제 6.32 합성곱 기반 문장 분류 모델 정의

```
import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(self, pretrained_embedding, filter_sizes, max_length, dropout=0.5):
        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
        embedding_dim = self.embedding.weight.shape[1]

        conv = []
        for size in filter_sizes:
            conv.append(
                nn.Sequential(
                    nn.Conv1d(
                        in_channels=embedding_dim,
                        out_channels=1,
                        kernel_size=size
                    ),
                    nn.ReLU(),
```

```

        nn.MaxPool1d(kernel_size=max_length-size-1),
    )
)
self.conv_filters = nn.ModuleList(conv)

output_size = len(filter_sizes)
self.pre_classifier = nn.Linear(output_size, output_size)
self.dropout = nn.Dropout(dropout)
self.classifier = nn.Linear(output_size, 1)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    embeddings = embeddings.permute(0, 2, 1)

    conv_outputs = [conv(embeddings) for conv in self.conv_filters]
    concat_outputs = torch.cat([conv.squeeze(-1) for conv in conv_outputs], dim=1)

    logits = self.pre_classifier(concat_outputs)
    logits = self.dropout(logits)
    logits = self.classifier(logits)
    return logits

```

예제 6.33 합성곱 신경망 분류 모델 학습

```

import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))

from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

```

```

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))

import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])

import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

```

```

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

from gensim.models import Word2Vec

n_vocab = len(token_to_id)
embedding_dim = 128

word2vec = Word2Vec.load("../models/word2vec.model")
init_embeddings = np.zeros((n_vocab, embedding_dim))

for index, token in id_to_token.items():
    if token not in ["<pad>", "<unk>"]:
        init_embeddings[index] = word2vec.wv[token]

embedding_layer = nn.Embedding.from_pretrained(
    torch.tensor(init_embeddings, dtype=torch.float32)
)

from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
filter_sizes = [3, 3, 4, 4, 5, 5]
classifier = SentenceClassifier(
    pretrained_embedding=init_embeddings,
    filter_sizes=filter_sizes,
    max_length=max_length
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if step % interval == 0:

```

```

        print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

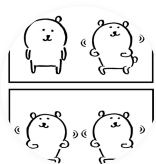
        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits)>.5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```



카사바

안녕하세요 :)



이전 포스트

Week10_예습과제

0개의 댓글

댓글을 작성하세요

댓글 작성

